```
d3.selectAll("path.domain").style("fill", "none").style("stroke", "black");
d3.selectAll("line").style("stroke", "black");
```

We'll want to be more specific in the future ("line.tick"), because it's likely that whatever we're working on will have more lines than those used in our axes.

We use selectAll because there are two of these paths, one for each axis we called.

If we set the `.orient()` option of the y-axis to `"left"` or the `.orient()` option of the x-axis to `"top"`, is seems like they aren't drawn. This is because they're drawn outside the canvas, like our earlier rectangles. To move our axes around, we need to adjust the `.attr("translate")` of their parent `<g>` elements, either when we draw them or later. This is why it's important to assign an ID to our elements when we append them to the canvas. We can move the x-axis to the bottom of this drawing easily:

```
d3.selectAll("#xAxisG").attr("transform","translate(0,500)");
```

Here's our updated code. It uses the `.tickSize()` function to change the ticks to lines and manually sets the number of ticks using the `ticks()` function:

```
var scatterData = [{friends: 5, salary: 22000},
          {friends: 3, salary: 18000}, {friends: 10, salary: 88000},
          {friends: 0, salary: 180000}, {friends: 27, salary: 56000},
          {friends: 8, salary: 74000}];

var xScale = d3.scale.linear().domain([0,180000]).range([0,500]);
var yScale = d3.scale.linear().domain([0,27]).range([0,500]);

xAxis = d3.svg.axis().scale(xScale)
          .orient("bottom").tickSize(500).ticks(4);
d3.select("svg").append("g").attr("id", "xAxisG").call(xAxis);

yAxis = d3.svg.axis().scale(yScale)
          .orient("right").ticks(16).tickSize(500);
d3.select("svg").append("g").attr("id", "yAxisG").call(yAxis);

d3.select("svg").selectAll("circle")
          .data(scatterData).enter()
          .append("circle").attr("r", 5)
          .attr("cx", function(d) {return xScale(d.salary);})
          .attr("cy", function(d) {return yScale(d.friends);});
```

Creates a pair of scales to map the values in our dataset to the canvas

Uses method chaining to create an axis and explicitly set its orientation, tick size, and number of ticks

Appends a <g> element to the canvas, and calls the axis from that <g> to create the necessary graphics for the axis

The effect all these functions is uninspiring, as shown in figure 4.8.

Let's examine the elements created by the axis code and shown in figure 4.8 as a giant black square. The `<g>` element that we created with the ID of `"xAxisG"` contains `<g>` elements that each have a line and text:

```
<g class="tick major" transform="translate(0,0)" style="opacity: 1;">
   <line x2="6" y2="0"></line>
   <text x="9" y="0" dy=".32em" style="text-anchor: start;">0</text>
</g>
```